

Тренажёр оператора 112 и диспетчера ДДС

Техническая документация

27 сентября 2026 г.

Содержание

Архитектура

Методы обработки данных

Данные и подбор служб

Место оператора 112

Место диспетчера ДДС

Кабинеты преподавателя и обучающегося

Руководство администратора

Установка и запуск

Безопасность и персональные данные

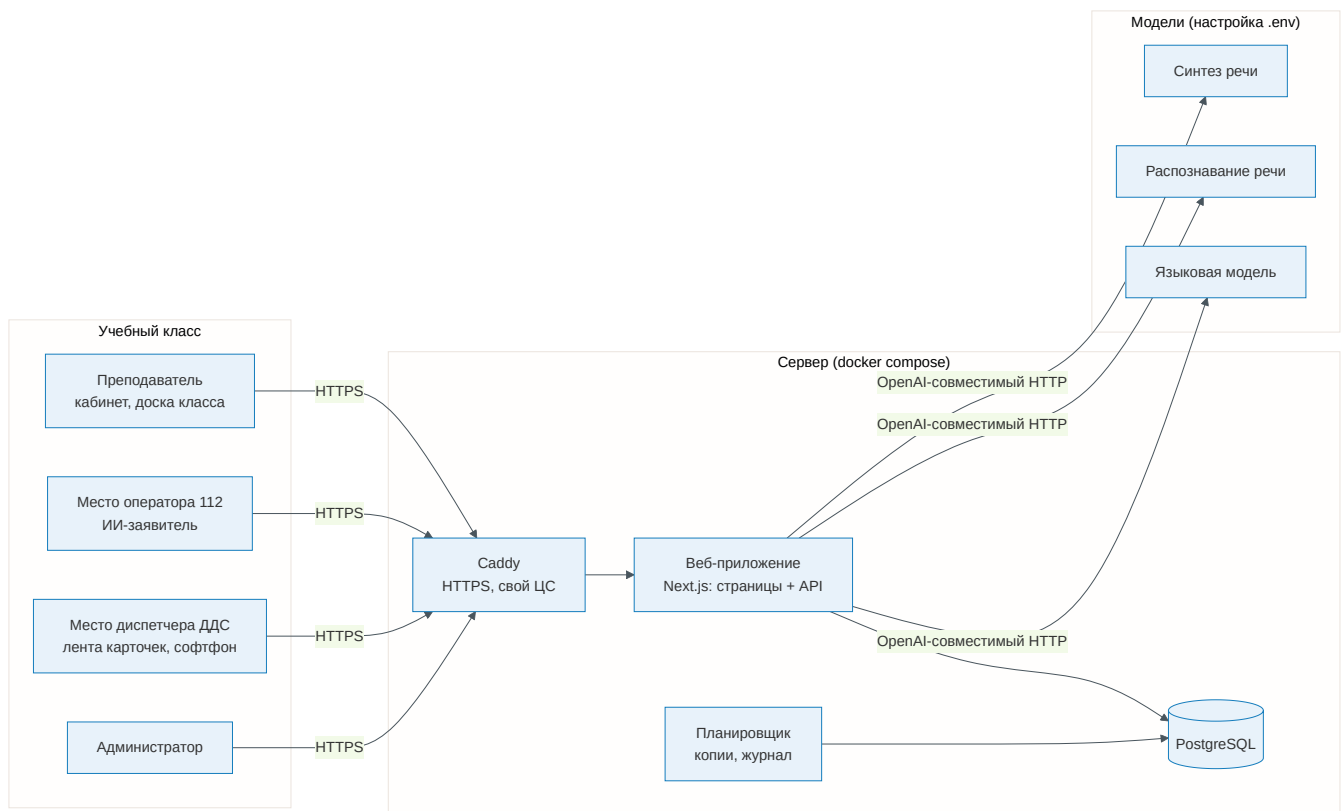
Производительность и нагрузка

Демо-стенд

Библиотеки и модели

Архитектура

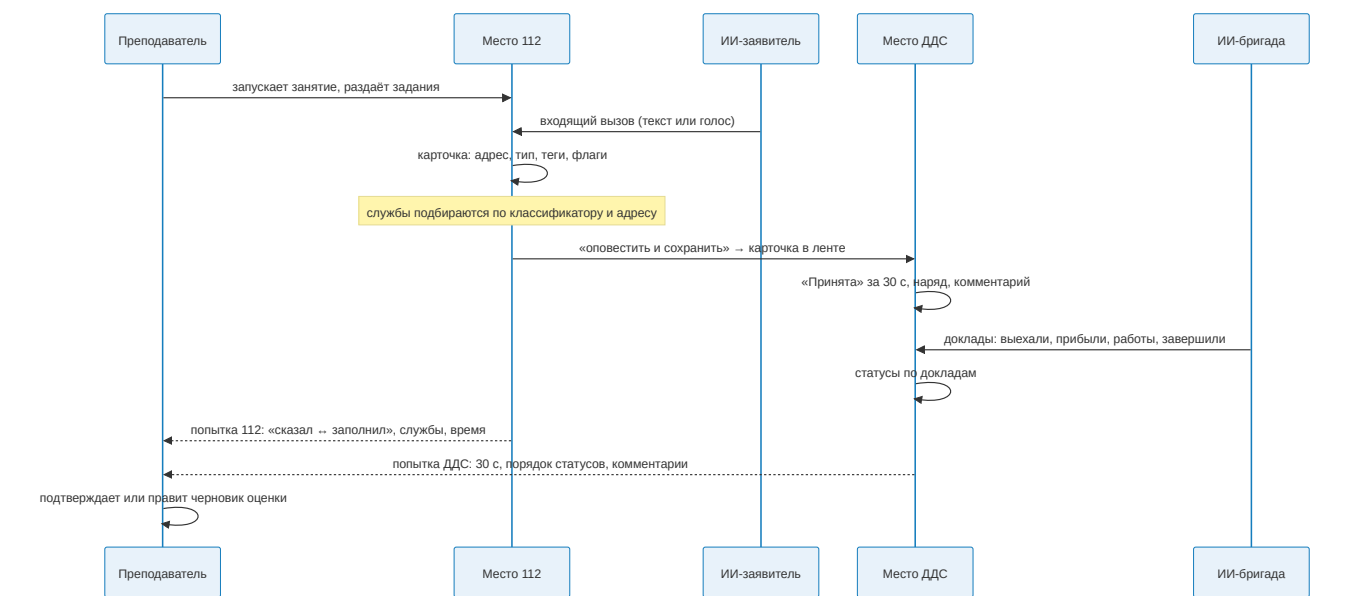
Компоненты



Одно веб-приложение обслуживает все роли. Рабочие места и доска класса опрашивают сервер раз в 1–2 секунды: без веб-сокетов, поэтому через любой прокси в контуре всё работает одинаково. Планировщик работает внутри процесса приложения: ежедневная резервная копия, ротация копий, срок хранения журнала.

Модели подключаются по OpenAI-совместимому HTTP API. Облако на стенде и локальные модели в контуре отличаются только строками в `.env`. Без модели система работает в режиме заглушки: заявитель отвечает по фактам сценария правилами.

Функциональная схема занятия



Данные

Группа	Таблицы	Откуда
Пользователи и безопасность	User, Session, AuditLog, Group	администратор, преподаватель
Справочники заказчика	IncidentGroup, IncidentType (1283 типа), Route (21 743 правила), Service (211 служб + скрытые получатели)	классификатор v046_24 и v046_11, список «СЛУЖБЫ 112» – см. data.md
Учебное содержание	Scenario	96 ситуаций из билетов, генерация ИИ, правка преподавателем
Занятие	Lesson, Seat, Incident, IncidentService, StatusEvent, Call	работа класса
Оценка	Attempt, WeightProfile	проверки правилами и ИИ, подтверждение преподавателем
Администрирование	SystemSetting, Backup	администратор, планировщик

Бизнес-данные – нормативы, веса оценки, справочники, сценарии – лежат в базе, а не в коде. Преподаватель и администратор меняют их без программиста.

Оценка

Каждая проверка возвращает результат одной из семи групп: время, порядок статусов, комментарии, адрес, тип и службы, полнота, понятность текста. Итоговый балл – взвешенная доля пройденных проверок. Проверки с ответом «не применимо» в знаменатель не входят, поэтому пустая попытка не получает 100 %. Критическая ошибка (например, улица-двойник) ограничивает балл 40 %. Сырые результаты проверок хранятся в попытке, поэтому смена весов мгновенно пересчитывает все попытки.

Масштабирование

Приложение не хранит состояние в памяти: сессии, очередь карточек и таймеры — в базе. Для роста можно поставить несколько копий приложения за Caddy и вынести PostgreSQL на отдельный сервер. Замеры нагрузки — в [performance.md](#).

Методы обработки данных

Сводка того, как тренажёр принимает решения: что считается правилами по данным заказчика, что делает языковая модель и как из этого получается оценка. Подробности — в документах по частям: data.md, dds.md, op112.md, teacher.md.

Главный принцип

Всё, от чего зависит правильность, считается правилами по данным заказчика: подбор служб по классификатору и адресу, допустимые переходы статусов, нормативы 30 секунд и 3 минуты, сверка карточки с эталоном. Языковая модель делает только то, что правилами не выразить: играет собеседника (заявителя, старшего бригады, службу), читает свободный текст преподавателя, готовит черновик пояснений к разбору.

Итоговую оценку ставит преподаватель. Черновик идёт в зачёт только после его подтверждения. Если модель недоступна (нет сети, кончился ключ, таймаут), система продолжает работать: собеседник отвечает по фактам сценария правилами, а проверки, которым нужна модель, помечаются «не применимо» и не влияют на балл.

1. Справочники заказчика

Источник	Что получаем	Как
Классификатор v046_24 (xlsx, 99 колонок, шапка в 3 строки)	24 группы, 1283 итоговых типа, 21 743 правила маршрутизации	<code>scripts/convert-classifier.py</code> : каждая колонка N-CU → пара «группа колонок, условие» (признак не выбран, угроза людям, пострадавшие, нет доступа, газификация, правонарушение, мед. помощь, эвакуация, перекрытие движения и др.)
Классификатор v046_11	«Сценарий реагирования» для 1260 типов	сопоставление по итоговому типу
«СЛУЖБЫ 112»	211 служб окна «Добавьте службы» + 17 скрытых получателей	<code>scripts/build-services.py</code> : короткое и полное имя, способ оповещения (АРМ-112, внешняя система, только телефон), округ и район
«КАРТОЧКА 112»	51 значение «что случилось», деревья тегов 101 / 104 / «Взрыв»	ручной разбор скриншотов
32 билета	96 ситуаций	15 проработаны вручную вместе с эталоном для ДДС, 81 — черновики

Подбор служб (`src/lib/routing/engine.ts`) — чистая функция от типа, флагов и адреса:

1. Для каждой плашки проверяются её колонки классификатора. Условная подколونка срабатывает, если выбран её признак и в ячейке есть значение, иначе работает «признак не выбран».
2. Территориальные ДДС добавляются по адресу: ДДС района или поселения и ДДС префектуры округа, только если адрес определён до района.

3. Главная служба (столбец М) ставится первой, остальные идут в порядке со скриншотов.

Проверено тестами на всех случаях со скриншотов заказчика и на его примере «пожар в школе в Щукино с пострадавшими» → 101, 102, 103, ЦЭМП, Деп. образования, ДДС Щукино, ДДС СЗАО.

2. Сценарии

- **Из билетов** (`scripts/build-scenarios.ts`). Персона заявителя: ФИО, роль, телефон, адрес, который он называет сразу, и скрытый адрес (курсив в билете), который выдаёт только на уточняющий вопрос; факты, которые сообщает только на вопрос. Эталон карточки: тип по классификатору, флаги, службы по движку подбора, обязательные вопросы. Эталон для ДДС: решение за 30 секунд, цепочка статусов, доклад бригады, что должно быть в итоговом комментарии, ловушки.
- **Из свободного текста** (`src/lib/scenarios/generate.ts`, экран «Сценарий из текста»). Эксперт описывает ситуацию своими словами – система собирает черновик. Модель, если подключена, извлекает персону, адрес, флаги и подсказку типа. Затем правила сопоставляют подсказку с листом классификатора: слова совпадают целиком или по основе, «без пострадавших» не путается с «с пострадавшими», лишние уточнения в названии дают штраф. Службы подбирает движок, район – справочник адресов, вопросы – классификатор и типовые вопросы группы.
 - Без модели текст читается правилами по ключевым словам. На 15 проработанных билетах правила определяют тип верно в 14 случаях (правила настраивались на этих же билетах), по всем 96 – в 39. Медицинские и прочие случаи без модели распознаются хуже, их уточняет модель или преподаватель.
- Любой сгенерированный сценарий – черновик. В занятие он попадает только после утверждения преподавателем, целиком или по разделам.

3. Собеседники на телефоне

Кто	Где	Правила поведения
Заявитель	место 112	говорит видимый адрес, истинный – только на уточняющий вопрос; факты – только если спросили; манера по характеру (паника, пожилой, ребёнок)
Старший бригады	место ДДС	сам звонит на каждом этапе: выехали, прибыли, работы, итог; отвечает, когда диспетчер звонит сам
Заявитель при перезвоне	место ДДС	отвечает по фактам сценария, номер карточки не знает

Промпты написаны под задачу 112 и ДДС. Без модели ответы строятся по фактам сценария: ключевые слова вопроса → нужный факт. Голос – нажми-и-говори: речь распознаётся сервером или браузером, ответ озвучивается мужским или женским голосом по персоне.

4. Оценка

Проверки. Каждая проверка возвращает результат: пройдена, провалена или не применима, с доказательством (время, цитата, статус) и правильным действием. Проверки разбиты на 7 групп: время, порядок статусов, комментарии и «кому передано», адрес, тип и службы, полнота, понятность текста.

- Место ДДС: 15 проверок по каталогу нарушений из памятки — ответ за 30 секунд, наряд за 3 минуты, статусы по докладам бригады не позже минуты, итоговый статус, итог в комментарии, «кому передано» при отказе, решение по эталону и др. Подробно — dds.md.
- Место 112: сверка «сказал заявитель ↔ заполнил оператор» с цитатами, адрес (улица-двойник — критическая ошибка), тип, флаги, службы, обязательные поля и вопросы, суть в первых 100 символах описания, время набора карточки. Подробно — op112.md.

Балл — взвешенная доля пройденных проверок:

балл = $\sum \text{по группам} (\text{вес группы} \times \text{пройдено в группе} / \text{применимо в группе}) / \sum \text{весов групп}$,
где есть применимые проверки

«Не применимо» в знаменатель не входит, поэтому пустая попытка не получает 100 %. Проваленная критическая проверка ограничивает балл 40 %. Сырые результаты хранятся в попытке, поэтому смена весов на панели преподавателя мгновенно пересчитывает все попытки. Жюри может само крутить веса.

Черновик → решение преподавателя. Преподаватель подтверждает черновик («Верно») или исправляет отдельные проверки с комментарием («ИИ неправ»). Правка пишется в журнал аудита: кто, когда, было, стало. В отчёты и кабинет обучающегося идут только подтверждённые попытки. Доля проверок, где преподаватель согласился с черновиком, показывается в отчёте — это мера качества автоматической оценки.

5. Отчёты и выводы по группе

Всё считается по подтверждённым попыткам: - время против норматива по каждому ученику; - лидеры и отстающие с причинами — какие проверки чаще проваливались; - типичные ошибки группы одной фразой («сегодня все валятся на адресе и на отказе без комментария»); - тепловая карта «ученик × группа проверок»; - уровни готовности по среднему баллу: 85 / 70 / 50.

Выгрузка — CSV для Excel.

6. Что сознательно не делает ИИ

- Не ставит итоговую оценку и не спорит с преподавателем.
- Не решает, какие службы должны быть в карточке: это классификатор заказчика.
- Не меняет утверждённые сценарии: переписанный ИИ раздел снова становится черновиком.
- Не видит персональных данных за пределами учебного занятия.

Данные и подбор служб

Тренажёр работает на тех же справочниках, что и Система-112 Москвы: классификатор происшествий, список служб, правила автоподбора служб и билеты учебного центра. Всё, что заказчик дал в таблицах и скриншотах, переведено в машиночитаемый вид и проверяется автотестами. Там, где материалы молчат или противоречат друг другу, решение принято явно и записано в `data/README.md`.

1. Откуда данные

Материал заказчика	Что взяли	Файл тренажёра
Классификатор v046_24 (xlsx, 1310 строк, 99 колонок)	24 группы, 1283 итоговых типа, признаки для кнопок оператора, вопросы оператору, главная служба, маршрутизация по 86 колонкам почти 60 организаций с условиями	<code>data/classifier.json</code>
Классификатор v046_11 (более старый)	колонок «Сценарий реагирования» — подтянута к 1260 типам по совпадению итогового типа	там же, поле <code>scenarioCode</code>
«СЛУЖБЫ 112» (55 скриншотов окна «Добавьте службы»)	все 211 служб: короткое имя для плашки, полное имя, округ и район для 157 территориальных ДДС	<code>data/services.json</code>
«КАРТОЧКА 112» (таблица и 39 скриншотов)	51 значение «Что случилось», частые кнопки, деревья тегов 101 / 104 / «Взрыв», реальные клики с автоподбором служб	<code>data/incident-kinds.json</code> , автотесты
Памятка ДДС, инструкция оператора 112	статусы и нормативы (30 секунд, «Не принята» только с комментарием, особенности 103 и 104), правило «район + округ»	эталон ДДС в сценариях
32 билета (96 ситуаций)	заявитель, ситуация, адрес со скрытым уточнением	<code>data/scenarios.json</code> , <code>data/addresses.json</code>

Исходные xlsx в репозиторий не входят: скрипты в `scripts/` пересобирают JSON из них одной командой, JSON лежат в репозитории и грузятся в базу командой `pnpm db:seed` примерно за две секунды.

2. Как устроен подбор служб

Оператор 112 выбирает «Что случилось», нажимает теги (где, что горит, угроза людям, нет доступа...) и вводит адрес. Тренажёр подбирает службы так же, как настоящая система, по одному правилу:

- Теги → итоговый тип.** Кнопки панели ведут на лист классификатора, например «Улица → Открытое пламя / Дым → Мусор» — это тип «пожар: мусор». Можно выбрать несколько листов: «квартира + газовая колонка + подъезд».
- Тип → маршруты.** У каждого типа в классификаторе заполнены ячейки маршрутизации: «служба реагирует» или «не реагирует». Многие службы разбиты на условные подколоники: Служба 101 — «нет доступа», МВД — «правонарушение» и «пострадавшие», ЦЭМП — «угроза людям», «пострадавшие»,

«мед. помощь», «эвакуация», МОСГАЗ — «газификация» и так далее.

3. **Флаги → условия.** Флаги карточки («Пострадавшие», «Нет доступа / Заблокированные») и ответы в панели («Угроза людям: Да») включают условия. В каждой группе колонок побеждает условная под колонка, если её признак выбран и служба в ней реагирует; иначе работает под колонка «признак не выбран».
4. **Адрес → территориальные ДДС.** Когда адрес определён до района, добавляются ДДС района (или поселения в ТиНАО) и ДДС префектуры его округа, а также «Автодороги» округа для дорожных типов. Пока в адресе только «Москва», территориальных служб нет — ровно как на скриншотах заказчика.
5. **Порядок.** Главная служба типа — первой, дальше порядок полосы служб со скриншотов (101, 104, 102, Деп. ЖКХ, ЦЭМП, ЦОДД, Мосгортранс, Мос.Без., Мослифт, ОАТИ...), затем ДДС района и ДДС префектуры.
6. **Невидимые службы.** Аппарат Мэра, ФСО, ГКУ НТУ и ещё 14 организаций получают карточки, но в полосе служб не показываются — так и в Системе-112.

Пример заказчика «пожар в школе в Щукино с пострадавшими»: теги «Здание / объект → Открытое пламя / Дым → Учебное заведение» дают тип «Пожар: учебное заведение», флаг «Пострадавшие» включает 103 и ЦЭМП, колонка «Департамент образования» даёт ДДС Деп. образования («по подчинённости»), адрес в Щукино — ДДС района Щукино и ДДС префектуры СЗАО. Всё, что назвал заказчик, тренажёр подбирает сам; дополнительно по классификатору приходят ФСБ, ЦОДД, Мос.Без., ОАТИ, ОЭК и Деп. ЖКХ.

Подбор — чистая функция `selectServices` в `src/lib/routing/engine.ts`: на входе типы, флаги и район, на выходе список служб с пометкой главной и причиной («пожар: мусор — угроза людям», «ДДС района по адресу: Щукино»). Причина показывается обучающемуся при разборе ошибок. Справочник читается из базы один раз и держится в памяти процесса.

3. Проверка на скриншотах заказчика

Автотесты (`src/lib/routing/engine.test.ts`) повторяют клики со скриншотов «КАРТОЧКА 112» и сверяют список плашек вместе с их порядком:

Что выбрано	Службы
Улица · пламя · нет доступа · мусор	Служба 101, ЦОДД, ОАТИ
+ угроза людям	+ ЦЭМП; «Нет» — ЦЭМП исчезает
+ правонарушение	+ Служба 102
+ газификация	101, 104, 102, ЦЭМП, ЦОДД, ОАТИ
Улица · запах гари	только Служба 101
Транспорт · пламя · нет доступа · общественный транспорт	101, 102, Деп. ЖКХ, ЦОДД, Мосгортранс, Мос.Без., ОАТИ
Дом · многоквартирный · квартира (газификация, угроза)	101, 104, 102, Деп. ЖКХ, ЦЭМП, ЦОДД, Мос.Без., Мослифт ...
Пожар в школе, Щукино, пострадавшие	101, 102, 103, ЦЭМП, Деп. образования, ДДС Щукино, ДДС СЗАО и др.
Запах газа в частном доме, поселение Вороновское	104, 101, ЦЭМП, Мособлгаз, ДДС Вороновское, ДДС ТиНАО

Все случаи совпадают со скриншотами. Единственное расхождение: на одном скриншоте ЦЭМП стоит без отмеченной угрозы — по классификатору ему нужна угроза, пострадавшие, мед. помощь или эвакуация. Тренажёр следует классификатору, расхождение описано в `data/README.md`.

4. Панели тегов и «Что случилось»

`data/incident-kinds.json` содержит список «Что случилось» (51 значение) с привязкой к группам классификатора, частые кнопки над полем и панели тегов:

- **101** — дерево «Где → Признак пожара → объект» по скриншотам (Улица, Транспорт, Дом, Здание / объект, Опасный объект) со строками «Доступ», «Угроза людям», «Медицинская помощь», «Эвакуация», «Газификация», «Правонарушение», «Перекрытие движения» — строки появляются по мере выбора, как в Системе-112;
- **104** и «**Взрыв**» — по признакам классификатора, подписи первого уровня как на скриншотах;
- **остальные группы** — общая панель «признак 1 → 2 → 3» и только те флаги, которые в этой группе реально меняют набор служб.

Хелпер `src/lib/routing/tags.ts` превращает выбор в панели в листья классификатора (итоговый тип и код), флаги и строку тегов для карточки.

5. Сценарии из билетов

Каждая из 96 ситуаций превращена в сценарий занятия (`data/scenarios.json`):

- **ИИ-заявитель:** ФИО, роль (очевидец, мама, соседка, работник АЗС...), телефон, адрес «как он его называет» и скрытый адрес, который выдаётся только на уточняющий вопрос (курсив в билете), ситуация, факты «только если спросят», манера речи и голос по полу;
- **эталон карточки 112:** итоговый тип и допустимые альтернативы, флаги, уточнённый адрес с районом и округом, службы (считает тот же движок), обязательные вопросы (из колонки J классификатора, по типу происшествия и по билету), ловушки;
- **карточка глазами ДДС:** строка тегов, «Класс.», флаги, адрес, описание, заявитель, службы;
- **сложность 1–10** по понятным признакам: скрытый адрес, другой регион, адрес-ориентир, число флагов и служб, паника заявителя, ловушки.

15 сценариев проработаны вручную и одобрены: живая речь заявителя и **эталон для ДДС** по ключевым службам — решение «Принята / Не принята» за 30 секунд (для «Не принята» — причина и кому передано), цепочка статусов по докладам старшего бригады, что обязательно в итоговом комментарии, ловушки из памятки (отказ «потому что реагирует другая служба», 103 не ставит «Не принята», 104 может не писать комментарии, дубли). Среди них билет 30-3 — ровно та ДДС «Поселение Вороновское», что на скриншотах заказчика. Остальные 81 — черновики с пометкой «ждёт проверки преподавателем»: преподаватель одобряет их в своём кабинете.

6. Адреса и похожие улицы

- `data/addresses.json` — 61 московский адрес из билетов с районом и округом (с оценкой уверенности) и 8 адресов, которые без точки на карте к району не привязать (участки МКАД, мосты).
- В билетах 25 ситуаций вне Москвы (Московская, Тульская, Рязанская, Владимирская области, Ржев): тренажёр записывает регион честно, территориальных ДДС Москвы не подбирает, для Московской области подсказывает «112 Мос. обл.». Есть и обратная ловушка: Крекшино звучит как область, а это Москва, ТиНАО.
- `data/confusable-streets.json` — 28 пар похожих улиц (Дубнинская / Дубининская, Белозерская / Беломорская, Ленинский / Ленинградский...). Хелпер `src/lib/routing/address.ts` сравнивает введённую улицу с эталоном и отличает «та же улица, записана иначе» от «похожая улица — критическая ошибка».

7. Принятые решения

Полный список с обоснованием — в `data/README.md`. Главные:

1. Условные колонки: «условие побеждает, иначе — признак не выбран» (буквальное чтение шапки теряло 101 и 102 в очевидных случаях).
2. Плашка «Деп. ЖКХ» — колонки «Деп. ЖКХ» и «Гор. Хозяйство»; для уличных типов — только когда известен район (так сходятся скриншоты 2026 и 2019 годов).
3. «Мособлгаз» — только для адресов ТиНАО.
4. Главная служба первой, порядок плашек — со скриншотов; места 103 и ФСБ — наше допущение.
5. Невидимые «слушатели» заведены отдельными службами с `visible = false`.
6. Районы адресов назначены вручную; для реальной эксплуатации нужна выгрузка ФИАС или OSM с полигонами районов.

8. Как обновить справочники

```
python3 scripts/convert-classifier.py <классификатор v046_24.xlsx> <классификатор v046_11.xlsx>
python3 scripts/build-services.py <список служб.json>
python3 scripts/build-incident-kinds.py
pnpm exec tsx scripts/build-scenarios.ts
pnpm test           # сверка со скриншотами и целостность сценариев
pnpm db:seed        # загрузка в базу, повторный запуск безопасен
```

Новый классификатор от заказчика подключается этими же командами: скрипты сами разбирают шапку, а если появилась незнакомая колонка или пропал тип, на который ссылаются панели и сценарии, сборка останавливается с понятной ошибкой.

Место оператора 112

Место 112 – учебная копия АРМ «Приём и обработка вызовов 112» по скриншотам заказчика. Обучающийся принимает вызов от ИИ-заявителя, расспрашивает его, заполняет карточку происшествия и оповещает службы. Сохранённая карточка сразу приходит на места диспетчеров ДДС того же занятия. После «отработана» система показывает разбор: совпадает ли то, что сказал заявитель, с тем, что записано в карточке, и верно ли выбраны службы.

Адрес: /ор112. Открыть может обучающийся, преподаватель и администратор; каждый видит только свои вызовы и карточки.

Как проходит вызов

1. **Ожидание.** Статус телефонии «Доступен», внизу журнал своих карточек с оценками и кнопкой «разбор».
2. **Входящий звонок.** Через несколько секунд всплывает синяя полоса «Входящий звонок с номера ... [Принять]» и звучит сигнал. Принять можно кнопкой, Enter или Insert.
3. **Карточка.** Открывается новая карточка «Происшествие N», таймер набора считает вверх и краснеет после норматива занятия (по умолчанию 65 с). АОН подставляется сам, канал связи определяется по номеру. Черновик сохраняется сам каждую секунду («Сохран.» в шапке), поэтому перезагрузка страницы ничего не теряет.
4. **Разговор.** Справа панель «Разговор с заявителем»: вопрос можно напечатать (Enter – сказать), выбрать из готовых фраз или сказать голосом (кнопка «Удерживайте, чтобы говорить» или пробел). Реплики заявителя звучат голосом; звук выключается кнопкой с динамиком. В учебном режиме под разговором видно, о чём уже спросили: адрес, пострадавшие, газ, этажность и т. д.
5. **Заполнение.** Блоки заполняются в любом порядке, как на АРМ:
 - заявитель: фамилия и имя (заглавные буквы ставятся сами), статус из 6 значений, канал связи, «вызов на иностранном языке»;
 - адрес: одна строка поиска с подсказками, поля заполняются сами, район и округ определяются сами; «очистить адрес» стирает всё;
 - описание со слов заявителя: счётчик 0 / 1999 и подсказка, что в службу 103 уходят первые 100 символов;
 - флаги «Пострадавшие», «Нет на месте / Отказ от скорой», «Нет доступа / Заблокированные»;
 - «что случилось?»: поиск по 51 типу с синонимами («пожар» находит 101), частые кнопки, значимые типы; можно выбрать несколько типов;
 - опросная карта: строки появляются по мере ответов, выбранная кнопка синяя. Под типами показан итоговый «Класс.».
6. **Службы.** Оранжевая полоса «Службы» собирается сама по классификатору, как только выбран тип и признаки; главная служба первая и подчёркнута. «+» открывает окно «Добавьте службы» (поиск по 211 службам, подобранные системой синие). Добавленную вручную службу можно убрать, подобранную системой – нельзя.
7. **Сохранение.** «сохранить» открывает окно «Список оповещаемых служб» с предупреждениями о незаполненных полях (как в оригинале, они не блокируют). «оповестить и сохранить карточку» регистрирует карточку, полоса служб темнеет, у плашек появляется «ЧЧ:ММ Добавлена», кнопка меняется на «отработана». Разговор на этом заканчивается.
8. **Разбор.** «отработана» закрывает карточку и открывает разбор. Следующий вызов приходит только после этого.

«нет контакта» и «срыв звонка» работают, пока тип не выбран: окно «Сохранить карточку как пустую?» и пустая карточка без служб. Если заявитель был на линии, разбор отметит это как критичную ошибку.

Горячие клавиши

Зажмите Alt – рядом с блоками появятся подсказки. Клавиши работают при любой раскладке.

Клавиши	Действие
Insert, Enter	принять входящий вызов
Alt+F1 / F2 / F3	телефоны: АОН / предоставленный / на место
Alt+Q	фамилия и имя заявителя
Alt+K	канал связи
Alt+A	строка адреса
Alt+P	флаг «Пострадавшие»
Alt+N	«нет контакта»
Alt+T	«что случилось?»
Alt+R	значимые типы происшествий
Alt+1...9	n-я опросная карта
Alt+Ctrl+1...9	n-я строка текущей опросной карты
Alt+O	описание со слов заявителя
Alt+Z	окно «Добавьте службы»
Alt+S	сохранить → оповестить и сохранить → отработана
Alt+V	важное происшествие
Alt+J	поле разговора с заявителем
Esc	закрыть окно

Alt+W, Alt+B и Alt+M (связи, напоминание, сообщение о проблеме) в учебной версии показывают подсказку. Alt+F1 и Alt+F2 в некоторых рабочих столах Linux забирает сама система – там к телефонам удобнее переходить мышью.

ИИ-заявитель

Заявитель – человек из учебного билета: ФИО, роль (очевидец, мама, сосед...), телефон, что случилось, адрес, как он его называет, и «скрытый» адрес, который выясняется только при уточнении, плюс факты билета («Дом 14 этажей, газифицирован»). Сценарии берутся из одобренных (`Scenario.status = APPROVED`): если преподаватель назначил месту свои задания, звонят по ним, иначе по одобренным сценариям категорий занятия; сначала те, что ещё не звонили, от лёгких к трудным.

Правила заявителя:

- в первой фразе говорит только, что случилось;

- называет адрес так, как в билете; точный адрес — только на уточняющий вопрос («какой номер дома?», «где именно?», повторный вопрос об адресе);
- факты (этажность, газ, доступ, пострадавшие, оружие...) — только если о них спросили; чего в билете нет — «не знаю»;
- манера по характеру: спокойный, в панике («Быстрее!», пожилой («Сынок, тут такое...»), ребёнок;
- заметки билета для преподавателя («сообщает только на вопрос», «при уточнении — дом 11») не произносит.

Два режима, переключаются настройкой модели (LLM_BASE_URL в .env):

- **с языковой моделью** — заявителя играет модель по промпту из `src/lib/op112/caller.ts`; модель возвращает реплику и ключи сказанных фактов;
- **без модели** — заявитель отвечает по правилам: вопрос оператора сопоставляется с темой (адрес, газ, этажность, пострадавшие...), а если тема не найдена — с фактом билета по общим словам («Какой номер маршрута?» → «Горит кабина автобуса маршрута № 63»). Ответы детерминированы, демо работает без сети.

В обоих режимах каждая реплика заявителя помечает, какие факты в ней прозвучали (`Call.messages[].revealed`). На этом держится проверка «сказал ↔ заполнил». Голос — общий модуль `src/components/voice` (сервер распознавания и синтеза или средства браузера).

Передача карточки в ДДС

«оповестить и сохранить карточку» создаёт то, что читает лента ДДС:

- `Incident` с `source = "op112"`, `lessonId` занятия, `status = "registered"`, `savedAt`; заявитель, адрес, флаги, `typeCodes` (листья классификатора, «Класс.»), `tags` (строка тегов опросной карты: `row, value`; текстовые строки и ответы «Да/Нет» — с названием строки: «Этажность здания: 14», «Угроза людям: Нет»), описание и запись журнала описаний «оп. N»;
- по плашке `IncidentService` в статусе `ADDED` (главная — `isMain`, `addedBy = auto | manual`) и событие `StatusEvent` «Добавлена» от «оп. 0».

Черновик до сохранения плашек не имеет, поэтому в ленту ДДС не попадает.

Оценка оператора 112

Оценка строится в `src/lib/op112/evaluate.ts` сразу после сохранения и хранится в `Attempt` (`kind = OP112`) как список проверок с доказательствами. Балл считается по активному профилю весов (`WeightProfile`) по группам `src/lib/scoring/score.ts`: каждая группа даёт долю пройденных проверок, группы складываются по весам, «не применимо» не входит в знаменатель, критичная ошибка ограничивает балл 40. Смена весов или правка преподавателем пересчитывает балл без повторной проверки.

Проверки по правилам (всегда):

Группа	Проверки
Время	карточка сохранена не позже норматива набора

Группа	Проверки
Адрес	улица, дом / корпус / строение, район, квартира / подъезд / этаж / код — против уточнённого адреса эталона. Известная похожая улица (Дубнинская вместо Дубининской) или то же название другого вида (улица вместо набережной) — критично ; опечатка — обычная ошибка
Тип и службы	тип «что случилось», «Класс.» (лист классификатора или допустимый), флаги эталона, недостающие и лишние службы
Полнота и вопросы	ФИО, статус, предоставленный телефон, описание; каждый обязательный вопрос эталона — по репликам оператора, с цитатой
Понятность текста	в первых 100 символах описания есть суть (пожар, газ, драка, объект) и пострадавшие, если они есть

«Сказал ↔ заполнил». Для каждого факта, который заявитель действительно произнёс, проверяется, что он есть в карточке:

- пострадавшие, угроза людям, газификация, доступ — флаги и кнопки опросной карты;
- этажность, «газ магистральный или баллон» — строки опросной карты;
- сознание, возраст — описание;
- ФИО, статус, телефон — поля заявителя.

Каждое расхождение показывается с цитатой: «Заявитель: «Дом 14 этажей, газифицирован» → в карточке «Проведена ли газификация»: не отмечено». Факт, о котором заявитель не говорил, в вину не ставится: тогда срабатывает проверка обязательного вопроса («не спросил про газ»). «Нет» в строке опросной карты засчитывается, только если оператор его выбрал: пустая строка — не «нет». Сведение, для которого в выбранной опросной карте нет строки, достаточно записать в описание. Факт считается сказанным, только если в реплике есть нужное значение (число этажей, слово про газ), а не просто похожие слова.

Проверки моделью (если модель настроена): модель читает расшифровку и карточку и находит сказанное, но не записанное, и оценивает, поймёт ли описание следующий диспетчер. Они выполняются после ответа сервера и дописываются в ту же попытку. Без модели эти две проверки «не применимо» и на балл не влияют.

Тренировка без занятия

Если у обучающегося нет занятия с местом 112, на /op112 есть кнопка «Тренировка без занятия». Она создаёт личное занятие с одним местом 112 и включёнными подсказками (в `Lesson.settings` отметки `practice`, `selfTraining` и `sessionId`). Вызовы идут по одобренным сценариям.

Практика привязана к сессии входа, а не только к учётной записи: на демо-стенде несколько проверяющих входят под одной демо-учёткой одновременно и не видят вызовы и карточки друг друга. После выхода или истечения сессии её практика завершается при следующем запуске тренировки. Занятие принадлежит преподавателю группы, поэтому попытки видны в его проверке; в списке занятий класса практика не показывается.

Если преподаватель завершил занятие во время вызова, открытую карточку можно дописать, сохранить и получить разбор; новых вызовов после этого нет.

Где что в коде

Путь	Что
<code>src/app/op112/ui/*</code>	экран: шапка, колонки, опросные карты, полоса служб, окна, разговор, разбор, горячие клавиши
<code>src/app/api/op112/*</code>	API: состояние места, тренировка, вызов, реплики, черновик, сохранение, отработана, пустая карточка, разбор, подбор служб, опросная карта, подсказки адреса
<code>src/lib/op112/catalog.ts, card.ts, panels.ts</code>	«что случилось», опросные карты поверх справочников, разбор карточки в листья классификатора, флаги и теги
<code>src/lib/op112/caller.ts, facts.ts</code>	ИИ-заявитель и факты билета
<code>src/lib/op112/evaluate.ts, review.ts</code>	оценка и попытка
<code>src/lib/op112/gazetteer.ts</code>	подсказки адреса без интернета: адреса билетов, пары похожих улиц, улицы Москвы

Тесты: `src/lib/op112/*.test.ts` (оценка, заявитель, опросные карты и службы на справочниках).

Место диспетчера ДДС

Место ДДС повторяет АРМ-112 дежурно-диспетчерской службы со скриншотов заказчика: лента происшествий, карточка в режиме просмотра, полоса служб, окно статуса и синее окно истории. Диспетчер карточку не проверяет (ответ заказчика #739) — он отвечает «Принята» или «Не принята» за 30 секунд, отправляет наряд и ставит статусы по докладам бригады, которые приходят по телефону (#691). Всё, что он сделал, сразу разбирается по нарушениям из памяти ДДС.

1. Как открыть

- Обучающийся входит и выбирает «Диспетчер ДДС» (адрес /dds).
- Если преподаватель запустил занятие и дал обучающемуся место с ролью ДДС и службой, лента сразу начинает получать карточки.
- Если занятия нет, на экране кнопка **«Тренировка без занятия»**: создаётся личное учебное занятие с местом «Поселение Вороновское» (профиль ДДС со скриншотов, билет Б30-3). Карточки приходят каждые 75 секунд, в очереди не больше двух, включены подсказки. Кнопка «Завершить тренировку» закрывает занятие и показывает разбор.
- Преподаватель и администратор могут открыть место обучающегося только для просмотра: /dds?seat=<id места> (преподаватель — только в своих занятиях).

2. Экран

Лента (/dds), как на скриншоте «Список происшествий»:

- шапка #EBCES с полем «Поиск происшествий» (ищет по номеру, адресу, типу, описанию и статусу) и кнопкой «сбросить»;
- тёмный блок #2F353A: дата в формате заказчика («Четверг, 17 Сентябрь 2026»), служба и ФИО на месте, крупные часы с секундами; время везде московское;
- тело #838F97: колонки Связи, закладка, ЧС, таймер, Опер. (бордовая ячейка #6C364B), АРМ, Номер, Дата, Время, Тип происшествия, Постр., Адрес, Статус службы; вторая строка «Описание» #49555D; новые сверху, 10 на странице, обновление раз в 1–1,5 секунды;
- **таймер** считает от «Добавлена» своей службы и краснеет (#FF0000) после 30 секунд без ответа; после ответа показывает, за сколько ответили (красным — если с опозданием). Норматив тикает у всех карточек в очереди сразу (#709);
- «v» открывает предпросмотр: статусы других служб, заявитель, информация;
- «уведомления» — сколько карточек ждут ответа; при новой карточке звучит короткий сигнал.

Карточка (/dds/incident/<номер>), раздел В3 разбора материалов:

- верхняя полоса: «Отключение», АОН, предоставленный номер, телефон на место (трубка рядом с номером звонит заявителю), «Происшествие N / Сохр. ...», кнопки «просмотр» и «дополнение»;
- слева: ФИО и статус заявителя, адрес жирным и описательный адрес, журнал описаний;
- справа: «Пострадавшие / Отказ от скорой / Заблокированные», ЧС и ЧП, тёмная шапка «Происшествие 101», строка тегов через точку, «Класс.: ... »;
- внизу полоса служб #49555D: плашки «название / ЧЧ:ММ последний статус», серая плашка — служба оповещается только по телефону, главная служба подчёркнута, время последнего статуса красное, если он поставлен с опозданием. Если служб больше восьми, второй ряд открывается кнопкой ⌵ (сам, если своя плашка во втором ряду);

- **своя плашка** с карандашом ✎: по нажатию карточка затемняется и внизу появляется строка «Статус ▼ | Номер наряда | Комментарий | ✓ | ✕», сама плашка синяя #157DBD. Номер наряда подставляется из прошлого статуса, подсказки — наряды службы;
- «^» над любой плашкой открывает синее окно истории: автор > дата время Статус > комментарий; опоздавший ответ выделен красным. Автор технических статусов — «оп. О», у внешних систем — «оп. 9999», у обучающегося — «Фамилия И О», как в памяти;
- «Получена службой» ставится сама, когда диспетчер впервые открывает карточку.

3. Статусы

Сейчас	Можно поставить
Добавлена, Получена службой	Принята, Не принята
Не принята	только Принята
Принята	Начало реагирования, Прибытие, Проведение работ, Отказ от выполнения работ, Работы завершены
Начало реагирования / Прибытие / Проведение работ	следующие по ходу работ (промежуточные можно пропустить), Отказ, Работы завершены
Работы завершены, Отказ	ничего: карточка закрыта для службы, ошибку исправляет отдел контроля

- Назад по статусам не ходят.
- Комментарий обязателен для «Не принята» и «Отказ» (причина и кому передано) и для «Работы завершены» (итоги — после сохранения карточка закрывается).
- **Служба 103** не ставит «Не принята» и «Отказ»: вместо них «Работы завершены» с комментарием «Завершение работ без бригады» — прямо с первого ответа.
- Первый ответ позже 30 секунд после «Добавлена» записывается с отметкой опоздания.
- Повторное нажатие или вторая вкладка не запишут статус дважды: статус меняется, только если его никто не изменил раньше.

Код: `src/lib/dds/status.ts` и тесты `status.test.ts`.

4. Как приходят карточки

Фоновых процессов нет: экран места раз в секунду опрашивает `/api/dds/state`, и этот опрос двигает поток (`src/lib/flow/dds-flow.ts`):

1. **Новая карточка** — если занятие идёт, у места открыто меньше `maxQueue` карточек и с прошлой прошло `tempoSec`. Первая приходит сразу. Сценарий: сначала задания, назначенные месту преподавателем, по порядку; иначе случайный одобренный сценарий из категорий занятия, без повторов, пока есть новые.
2. **Плашки** берутся из карточки сценария. Своя служба места есть всегда (#684, один алгоритм для всех ДДС): если места нет в карточке, районная ДДС или ДДС префектуры встаёт на место плашки своего уровня и получает её эталон; остальные службы добавляются в конец.
3. **Карточки, сохранённые на местах 112** того же занятия, тоже приходят в ленту, если среди их плашек есть служба места.

4. **Чужие плашки живые:** внешние системы получают карточку за секунды, службы на АРМ — когда «их диспетчер» откроет; дальше «Принята» с номером наряда, выезд, прибытие, работы, «Работы завершены» с комментариями. Серые плашки (только телефон) статусов не получают. Расписание у каждой плашки своё, но одинаковое при каждом опросе (`src/lib/dds/bots.ts`).

Настройки занятия (`Lesson.settings`): `tempoSec`, `maxQueue`, `ackSec` (30 с), `workSec` (3 мин), `cardSource`, `categories`, `brigadeReports`, `hints`; `practice` отмечает самостоятельную тренировку.

5. Телефон

Софтфон открывается кнопкой «Телефон» справа внизу и сам всплывает при входящем звонке. Работает текстом; голос включён по умолчанию: реплики собеседника звучат мужским или женским голосом персонажа, ответить можно кнопкой «нажми и говори» (или пробелом), текстовое поле остаётся запасным вариантом. Всё сказанное пишется в `Call.messages`.

- **Наряд назначается** номером наряда в статусе («Принята», наряд 23) или по телефону: позвонить свободному наряду из книжки и сказать, куда ехать. У каждой службы три наряда со старшими (`src/lib/dds/personas.ts`).
- **Доклады (#691, вариант «а»):** после назначения наряд проходит этапы из эталона сценария — выехали, прибыли, начали работы, закончили (или «работы проводить не будем») — за время `workSec`. На каждом этапе старший звонит сам; звонок звонит 25 секунд и, если не ответить, становится пропущенным.
- **Звонок наряду (вариант «б»):** диспетчер сам звонит старшему и спрашивает ход работ — тот докладывает, где наряд сейчас.
- **Перезвон заявителю (#739–740):** трубка у номера в карточке или книжка. Заявитель отвечает только по фактам сценария; номер карточки он не знает, и если диспетчер его назовёт, это попадёт в разбор.
- **Другие службы карточки и организации из эталона** (управляющая компания, «Ростелеком») — из книжки; у служб без номера в справочнике учебный номер (101–104 — короткие).
- **112** — звонок по памятке, когда обстановка изменилась: оператор просит представиться, назвать адрес, номер карточки и что изменилось.
- **Журнал** — все звонки с расшифровкой, пропущенные красным.

Без адреса языковой модели (`LLM_BASE_URL`) собеседники отвечают детерминированно по фактам сценария; с моделью — по коротким промптам из `personas.ts`, а при ошибке модели всё равно звучит заготовленная реплика.

6. Разбор

Плашка разбирается, когда получила «Не принята», «Работы завершены» или «Отказ», и ещё раз в конце занятия (карточки без ответа тоже). Результат — `Attempt` с `kind = DDS`: список проверок с доказательствами (время, цитаты) и балл по активному профилю весов. Проверенный преподавателем разбор больше не перезаписывается.

Проверка	Группа весов	Что ловит (памятка ДДС, D4)
Ответ «Принята / Не принята» за 30 с	время	нет ответа вообще — критично (карточка «Не оповещено»); ответ позже норматива

Проверка	Группа весов	Что ловит (памятка ДДС, D4)
Наряд направлен в пределах отработки (3 мин)	время	«Принята» без наряда; не применяется, если служба по эталону наряд не высылает
Звонки наряда приняты	время	пропущенные доклады
Статус поставлен сразу после доклада	время	статус позже 60 с после доклада или не поставлен
В «Не принята» / «Отказ» указана причина	комментарии	пустая отговорка
Указано, кому передана информация	комментарии	нет организации, «передано», «дубль», «КП №»
Итоговый комментарий содержательный	комментарии	«Работы завершены» без итогов
В комментарии есть то, что требует эталон	комментарии	не хватает пунктов <code>commentMustHave</code> (сверка по основам слов, нужна половина)
Статусы хода работ проставлены	порядок статусов	сразу «Принята → Работы завершены», хотя бригада выезжала
Статусы ставятся по факту	порядок статусов	«Прибытие» или «Работы завершены» раньше доклада
Статус соответствует смыслу	порядок статусов	«Принята: не обслуживаем», «Работы завершены: нет договора»
Карточка закрыта правильным статусом	порядок статусов	«Работы завершены» там, где по эталону «Отказ»
Решение совпадает с эталоном	службы	отказ от профильного происшествия – критично
Перезвон без номера карточки	полнота	номер карточки назван заявителю
Комментарии понятны следующему диспетчеру	грамотность	оценивает преподаватель, пока модель не подключена

Проверка, которая к карточке не относится, получает «не применимо» и не входит в балл: карточка, где проверять нечего, не получает 100 %. При провале критичной проверки балл не выше 40. Код: `src/lib/dds/evaluate.ts` (правила и тесты), `src/lib/dds/review.ts` (сбор фактов и запись).

Обучающийся видит разбор на своём месте: во время самостоятельной тренировки – сразу, на занятии – после его завершения («Разбор»: балл, ошибки, доказательства и «как правильно»).

7. Что берётся из сценария

Сценарий в формате `data/scenarios.json`:

- `ddsCard: classLabel` → «Класс.», `tagsLine` → строка тегов, `flags` (в том числе «Заблокированные»), `address` и `descriptive, description, caller, services` (имена плашек); `truth.kind` → «Происшествие 101», `truth.address` и `truth.services` – адрес и плашки с номерами служб.

- `ddsReference.services[]` для службы места (или для её роли): `decision` (ACCEPTED / REJECTED), `decisionComment`, `chain` (этапы бригады), `brigadeReport` (итоговый доклад), `commentMustHave`, `traps`.
- Необязательные поля, которые понимает место ДДС: `contacts` — номера организаций для книжки, `transferTo` — слова «кому передано» для проверки, `crew.work` и `crew.refuse` — что говорит бригада на месте и почему отказывается.

Для разработки есть четыре дополнительных сценария Вороновского (пожар в квартире, лифт у подрядчика, прорыв трубы, провод «Ростелекома»): `pnpm exec tsx prisma/dev-fixtures-dds.ts` после `pnpm db:seed`.

8. Для других частей системы

- API места: `GET /api/dds/state` (опрос и поток), `GET /api/dds/feed`, `GET /api/dds/incidents/<номер>`, `POST /api/dds/incidents/<номер>/status`, `POST /api/dds/calls`, `POST /api/dds/calls/<id>/answer | say | hangup`, `POST /api/dds/practice`, `POST /api/dds/practice/finish`, `GET /api/dds/results`. Каждый маршрут проверяет вход; обучающийся видит только своё место и свои карточки, запись — только на своём месте в идущем занятии.
- Кнопка преподавателя «Завершить занятие» должна вызвать `finishLessonEvaluation(lessonId)` из `src/lib/dds/review.ts` (вызов можно повторять). Если не вызовет, разбор всё равно появится, когда обучающийся откроет место.
- Сгенерированная карточка знает своё место: `Incident.ddsSeatId`. Что видит место — `seatFeedWhere(seat)` из `src/lib/dds/scope.ts`.
- Карточка с места 112 попадает к ДДС, когда её `status` не `draft` и среди плашек есть служба места. Если на занятии несколько мест ДДС с одной службой, карточку 112 видят все они и работают с одной плашкой — местам ДДС лучше давать разные службы.

Кабинеты преподавателя и обучающегося

Сегодня преподаватель учебного центра ходит по классу и смотрит каждому через плечо. В тренажёре он видит весь класс с одного экрана: кто открыл карточку, у кого горит таймер, кто сдал и где ошибся. Проверки и ИИ готовят черновик оценки, решение всегда за преподавателем: без его подтверждения попытка не идёт в зачёт и в отчёты.

Преподаватель входит под своей учётной записью и попадает на страницу «Занятия». Разделы кабинета: **Занятия, Сценарии, Веса оценки, Отчёты, Группы**. Кабинет работает на компьютере, планшете и телефоне.

Демо-учётки и демо-занятия: `pnpm db:seed`, затем `pnpm exec tsx prisma/seed-demo.ts` (с флагом `--live` — ещё и идущее занятие с тикающими таймерами). Преподаватель — `teacher / Teacher2026`, обучающиеся — `student1 ... student5 / Student2026`.

1. Занятие

Создание — кнопка «Новое занятие»:

- название и группа (видны только свои группы; группы и их состав ведёт администратор);
- **категории** происшествий — из утверждённых сценариев; ничего не выбрано — все утверждённые;
- **источник карточек**: сгенерированные (карточки на места ДДС готовит система), сформированные учениками (приходят с мест 112), смешанные;
- **темп** (новая карточка раз в N секунд), размер очереди на месте;
- **нормативы**: ответ ДДС «Принята / Не принята» (30 с), обработка карточки (3 мин), набор карточки 112 (1:05) — по умолчанию из настроек администратора;
- **режим подсказок** для начинающих и **доклады наряда** по телефону.

Места и задания. Каждому ученику группы — место: роль «112» (принимает вызовы ИИ-заявителя и заполняет карточку) или «ДДС» (получает карточки и ведёт статусы). Для ДДС выбирается служба, по умолчанию «Поселение Вороновское» — как на скриншотах заказчика. Задания раздаются галочками по местам — «первому — задание 1, пятому — задание 3»: у каждого места ряд номеров, номер = утверждённый сценарий из списка над таблицей. Быстрые кнопки: «Всем ДДС», «Всем 112», «Чередовать», «Задания по кругу», «Все задания всем». Режим **«Одна карточка на всех»** даёт всем местам одни и те же задания — удобно сравнить учеников на одной ситуации. Место без заданий получает карточки из выбранных категорий. Отсутствующего ученика снимают галочкой.

В задание попадают только утверждённые сценарии: всё сгенерированное остаётся черновиком, пока его не проверит преподаватель.

Старт и стоп. Занятие создаётся черновиком, его можно править и удалить. «Начать занятие» запускает поток карточек; с этого момента места, задания и настройки заморожены, а оценки занятия нельзя подтверждать и править — ничего не меняется посреди работы класса. Ученик не может быть на двух идущих занятиях сразу (самостоятельная тренировка на рабочем месте не мешает — занятие преподавателя её перекрывает). «Завершить занятие» останавливает поток в любой момент. «Провести ещё раз» создаёт копию с теми же местами и заданиями. Старт, стоп, правка и удаление пишутся в журнал аудита (кто, когда, было, стало).

2. Живая доска класса

Экран занятия. Обновляется раз в 2 секунды, таймеры идут каждую секунду.

На каждое место — плитка:

- номер места, роль (112 — оранжевая метка, ДДС — синяя), служба, ФИО;
- **текущая карточка**: номер, ситуация, адрес, статус своей службы;
- **таймер** с нормативом: у ДДС — «до ответа» (30 с), затем «обработка» (3 мин) до отправки наряда; у 112 — набор карточки. Просрочка — красным, плитка обводится красным;
- **открыл / ответил / сдал** (у 112 — вызовов / сохранил / сдал) и очередь карточек;
- красные метки статусов карточки, как у отдела контроля Системы-112, и ошибки по проверкам с самыми частыми.

Статус карточки	Когда	Цвет
Не оповещено	служба не ответила «Принята» / «Не принята» за норматив — в том числе, пока карточка ждёт в очереди	красный
Отказ	«Не принята» или «Отказ от выполнения работ»	красный
Не завершено	занятие закончилось (в жизни — прошло 48 ч), а «Работы завершены» нет	красный
Заполняется, Зарегистрирована, Завершена	обычный ход	серый

Статусы считаются только по плашкам, которые ведут места занятия: службы, которые играет система, ученикам не засчитываются. Карточка с места 112, которая ушла сразу нескольким местам ДДС одной службы, стоит в очереди у каждого, пока кто-то её не возьмёт, и никому не приписывается как просрочка.

Под плитками — все карточки занятия: номер, время, место 112, адрес, плашки служб с местом и статусом, итоговый статус; красные — сверху.

Режим проектора — кнопка « Режим проектора» (адрес ?projector=1): тёмный экран на всю стену, крупные плитки с таймером, ФИО и красными метками, без лишнего. Для внешнего монитора или проектора в классе.

3. Проверка попыток

Попытка — работа одного места над одной карточкой: карточка, заполненная на месте 112, или плашка своей службы на месте ДДС. Рабочие места проверяют попытку правилами (время, порядок статусов, комментарии, адрес, службы, полнота), а понятность текста и «сказал ↔ заполнил» — моделью ИИ. Каждая проверка хранит вердикт, доказательство («Принята» через 0:50 после «Добавлена», «Заявитель: “Дубнинская”, в карточке: “Дубининская”») и правильный ход.

Список попыток занятия — с фильтрами: на проверке / подтверждены / исправлены, роль, место. Экран попытки:

- черновик оценки: итог и советы (по правилам, а если подключена модель — кнопка «Подготовить черновик» просит её объяснить вердикты простыми словами; менять вердикты модели запрещено);
- проверки по группам весов: вердикт, «правило» или «оценил ИИ», критичность, доказательство, «как надо»;

- ход работы: статусы плашки ДДС со временем от «Добавлена» и опозданиями, или разговор с заявителем и карточка 112;
- черновой балл и итоговый балл.

Решение преподавателя:

- **«Верно — подтвердить»** — черновик верен, попытка идёт в зачёт;
- **«ИИ неправ — исправить»** — у каждой проверки переключатель «Верно / Ошибка / Не применимо» и обязательный комментарий ученику «как надо». Балл пересчитывается сразу, ещё до сохранения. Сохраняются только те проверки, которые преподаватель изменил;
- **«Изменить решение»** и **«Вернуть на проверку»** — если ошибся.

Каждое решение пишется в журнал аудита: кто, когда, статус и балл до и после, какие проверки и как изменены, комментарий. Пока занятие идёт, решения закрыты. После решения — «Следующая на проверке».

4. Веса оценки

Балл попытки — от 0 до 100. Проверки разбиты на семь групп, у каждой группы вес от 0 до 5:

Группа	Что внутри	По умолчанию
Время	ответ ДДС за 30 с, наряд за 3 мин, набор карточки 112	3
Порядок статусов	статусы по порядку, доклады наряда, закрытие карточки	2
Комментарии и «кому передано»	причина и кому передано у «Не принята» и «Отказа», итог работ	2
Адрес	улица и дом; похожая улица — критичная ошибка	3
Тип происшествия и службы	тип, службы, решение принять или отказать	3
Полнота карточки и вопросы	заявитель, пострадавшие, обязательные вопросы, «сказал ↔ заполнил»	1
Понятность текста	понятно ли следующему диспетчеру	1

В каждой группе считается доля пройденных проверок, группы складываются по весам. «Не применимо» в расчёт не входит, поэтому попытка, где нечего проверять, не получает 100. Критичная ошибка (похожая улица, отказ от профильного происшествия) ограничивает балл сорока.

Панель «Веса оценки»: двигаете ползунок — и сразу видно «было → стало» до сохранения: средний балл, сколько попыток изменили балл, сколько набрали 70 и больше, средние по ученикам (шкала 0–100), по занятиям и самые большие сдвиги. «Сохранить и пересчитать» делает профиль активным и пересчитывает баллы всех попыток по сохранённым сырым результатам проверок — точно, а не приблизительно. Веса общие для центра; пока идёт занятие, сохранить их нельзя. Изменение весов пишется в аудит.

5. Отчёты

«Отчёты» — список проведённых занятий; отчёт открывается и с экрана занятия. В отчёт идут только подтверждённые попытки, про остальные — сколько ждут проверки и ссылка на них.

- **Ученики:** ФИО, место и роль, число попыток, среднее время (набор карточки у 112, ответ «Принята» у ДДС), норматив места, отличие от норматива, опоздания, действия, ошибки, ошибки в тексте, критичные, средний балл, уровень готовности (от «готов к самостоятельной работе» до «нужен разбор с преподавателем»).
- **Лидеры и отстающие — и почему:** для лидеров — в каких группах нет ошибок и что подтянуть, для отстающих — самые частые ошибки, критичные, опоздания.
- **Типичные ошибки группы** и вывод одной фразой: «Сегодня группа чаще всего ошибается: ...».
- **Согласие преподавателя с черновиком:** какую долю проверок преподаватель оставил как есть, отдельно — проверки ИИ. Это сверка «рекомендации сервиса против реальных».
- **Графики** без внешних библиотек: средний балл, время против норматива, доля провалов по проверкам и тепловая карта «ученик × группа проверок». Шкалы от нуля, подписи и единицы на месте, легенда цветов.
- **CSV:** по ученикам и по попыткам. UTF-8 с BOM и точкой с запятой — открывается двойным щелчком в Excel с русскими настройками. Выгрузка пишется в аудит.

6. Группы

История успеваемости: для каждой группы — средний подтверждённый балл каждого ученика на последних занятиях и изменение от первого к последнему.

7. Сценарии

Библиотека сценариев общая для преподавателей центра: 96 ситуаций из билетов заказчика плюс то, что сгенерирует ИИ. Фильтры по статусу (черновики, утверждённые, архив), категории и поиску по названию или номеру билета.

Сценарий состоит из четырёх разделов, каждый утверждается отдельно:

1. **Заявитель** — кто звонит, что говорит сразу, какой точный адрес называет только на уточняющий вопрос, факты, характер и голос.
2. **Эталон карточки 112** — тип, теги, признаки, адрес, службы, обязательные вопросы, ловушки.
3. **Карточка на месте ДДС** — что видит диспетчер.
4. **Эталон действий ДДС** — решение по каждой службе, цепочка статусов, что должно быть в комментарии, ловушки. Правильный ход подсвечен.

Сценарий готов к занятиям, когда утверждены все его заполненные разделы («Утвердить целиком» — одной кнопкой). Заявителя правят формой, остальные разделы — как JSON с проверкой формата. «Исправь...» с замечанием отправляет раздел модели ИИ: она переписывает его, раздел возвращается черновиком, замечание сохраняется в истории сценария. Без модели (режим заглушки) тренажёр прямо говорит, что регенерация недоступна, и сохраняет замечание. «В архив» убирает сценарий из занятий, не ломая прошлые результаты. Пока сценарий может выпасть в идущем занятии, его нельзя править, снимать утверждение и убирать в архив. Все действия — в журнале аудита.

8. Кабинет обучающегося

«Мои результаты»:

- сводка: проверено попыток, средний балл, последнее занятие и сдвиг к прошлому;
- прогресс по занятиям (шкала 0–100);
- «Что подтянуть» — три самых частых группы ошибок с советом и конкретными примерами «как надо»;
- список попыток. Подтверждённая — балл и разбор по проверкам с комментарием преподавателя. Неподтверждённая — только «на проверке»: ни балла, ни черновика ИИ ученик не видит.

Ученик видит только свои попытки: каждый запрос к данным фильтруется по его учётной записи, чужая попытка отвечает «не найдено» — так же, как несуществующая. Это закрыто автотестом `tests/student-isolation.test.ts`; изоляция преподавателей (чужие занятия, доска, отчёт, проверка — «не найдено») — `tests/teacher-isolation.test.ts`.

9. Для разработчика

- Страницы: `src/app/teacher/*`, `src/app/student/results/*`.
- API: `src/app/api/teacher/*` (занятия, старт/стоп/копия, доска, попытки, решение по попытке, черновик, веса, отчёт CSV, сценарии) и `src/app/api/student/*`. Каждый обработчик начинает с проверки сессии и роли; данные читаются через ограничения по владельцу (`src/lib/teacher/access.ts`).
- Логика без базы, покрыта тестами: доска `src/lib/board/state.ts`, решение по попытке `src/lib/review/review.ts`, отчёт `src/lib/reports/lesson.ts`, CSV `src/lib/reports/csv.ts`, кабинет ученика `src/lib/student/results.ts`, утверждение сценариев `src/lib/scenarios/sections.ts`.
- Контракт с рабочими местами: они пишут `Attempt.criteria` как список `CriterionResult` (`src/lib/scoring/score.ts`), а балл считает только `computeScore`. Доска читает карточки, плашки служб, события статусов (с местом, которое действовало), звонки и попытки; место ДДС для сгенерированной карточки — `Incident.ddsSeatId`.

Руководство администратора

Администратор входит под своей учётной записью и попадает на страницу «Состояние».

Состояние системы

Обновляется раз в 5 секунд: база данных и время отклика, нагрузка процессора, память, свободное место на диске, активные сессии, идущие занятия, ошибки сервера за последний час, последняя резервная копия, какие модели ИИ подключены. Красная рамка — повод вмешаться.

Проверка для внешнего мониторинга без входа: `GET /api/health → {"ok":true}` (данных не раскрывает).

Пользователи

- Создание учётной записи: логин, ФИО, роль (администратор, преподаватель, обучающийся), пароль по политике — не короче 8 символов, буквы и цифры.
- Блокировка и разблокировка. Заблокированный пользователь сразу теряет все сессии.
- Смена пароля. Все сессии пользователя завершаются.
- После 5 неудачных входов подряд учётная запись закрывается на 15 минут (настраивается в `.env: MAX_FAILED_LOGINS, LOCK_MINUTES`). Срок сессии — `SESSION_HOURS`.
- На демо-стенде (`DEMO_MODE=true`) демо-учётки защищены от блокировки и смены пароля, чтобы проверяющие не мешали друг другу.

Журнал аудита

Каждое значимое действие записывается: входы и неудачные попытки, блокировки, изменения пользователей и настроек, подтверждение и правка оценок преподавателем (было → стало), резервные копии, ошибки сервера. Фильтры по событию, пользователю и датам, выгрузка в CSV для Excel.

Журнал хранится не меньше 6 месяцев: срок задаётся в настройках, но ниже 183 дней система его не опускает. Старые записи удаляет встроенный планировщик раз в час.

Резервные копии

- Автоматически каждый день в заданное время (по умолчанию 03:00), хранятся заданное число дней (по умолчанию 14).
- Вручную — кнопка «Сделать копию сейчас».
- Формат — `pg_dump custom`, каталог задаётся `BACKUP_DIR` (в Docker — том `backups`).

Восстановление выполняется из консоли сервера, когда не идёт занятие:

```
## Docker
docker compose exec app sh scripts/restore.sh /backups/trainer-2026-09-27T03-00-00.dump

## без Docker
DATABASE_URL=... scripts/restore.sh backups/trainer-2026-09-27T03-00-00.dump
```

Кнопки восстановления в интерфейсе нет намеренно: случайный клик не должен стереть идущее занятие.

Настройки

Нормативы (30 с на ответ ДДС, 3 мин на отработку, таймер набора карточки 112, 48 ч до «Не завершено»), срок хранения журнала, время и срок хранения резервных копий. Каждое изменение пишется в журнал. Адреса моделей, база данных и параметры входа задаются при установке в `.env`.

Установка и запуск

Три способа: для разработки, в Docker на одном сервере, офлайн в изолированном контуре учебного класса. Во всех случаях модели ИИ подключаются настройкой в `.env`, код менять не нужно.

Требования

Что	Минимум
Сервер	Ubuntu 20.04+ или Windows 10/11 с Docker Desktop; 4 ядра, 8 ГБ ОЗУ, 20 ГБ диска (без локальных моделей)
Локальные модели (по желанию)	+ 16 ГБ ОЗУ для языковой модели 7B на процессоре или видеокарта от 8 ГБ
Рабочие места	Chrome, Firefox или Яндекс.Браузер; для голоса – гарнитура и HTTPS

1. Разработка

```
cp .env.example .env
pnpm install
pnpm db:up && pnpm db:migrate && pnpm db:seed
pnpm dev # http://localhost:3100
```

2. Docker на одном сервере

```
cp .env.example .env # при необходимости задайте адреса моделей
docker compose --profile app up -d --build
## приложение: http://<сервер>:3000, демо-учётки – в README
```

При старте контейнер сам применяет миграции и загружает демо-учётки и справочники (повторный запуск ничего не дублирует). У всех сервисов `restart: always` и проверка здоровья: после сбоя или перезагрузки сервера всё поднимается само.

HTTPS

Браузер даёт доступ к микрофону только по HTTPS, поэтому для голосовых звонков нужен профиль `https` (Caddy перед приложением):

```
## публичный стенд с доменом – сертификат Let's Encrypt выпускается автоматически
SITE_ADDRESS=trainer.example.ru docker compose --profile app --profile https up -d

## учебный класс без интернета – собственный центр сертификации Caddy
SITE_ADDRESS=192.168.1.10 TLS_MODE="tls internal" docker compose --profile app --
profile https up -d
docker compose cp caddy:/data/caddy/pki/authorities/local/root.crt ./root.crt
```

root.crt один раз устанавливается на компьютеры класса в «Доверенные корневые центры сертификации» (Windows: двойной щелчок → «Установить сертификат» → «Локальный компьютер»). После этого адрес https://192.168.1.10 открывается без предупреждений и с микрофоном.

3. Офлайн-установка (без интернета)

На машине с интернетом:

```
docker compose --profile app build
docker pull postgres:16-alpine && docker pull caddy:2-alpine
docker save trainer-112-dds:latest postgres:16-alpine caddy:2-alpine | gzip >
trainer-images.tar.gz
```

Перенести в контур trainer-images.tar.gz и папку проекта, затем:

```
gunzip -c trainer-images.tar.gz | docker load
docker compose --profile app --profile https up -d
```

Модели ИИ

Все три сервиса говорят на OpenAI-совместимом HTTP API, поэтому облачный и локальный вариант отличаются только строками в .env:

Назначение	Переменные	Облако (стенд)	Локально (контур)
Языковая модель	LLM_BASE_URL, LLM_MODEL, LLM_API_KEY	любой OpenAI-совместимый API	Ollama или llama.cpp: http://<хост>:11434/v1, модель Qwen2.5 7B Instruct
Распознавание речи	STT_BASE_URL, STT_MODEL	облачный STT	faster-whisper-server
Синтез речи	TTS_BASE_URL, TTS_MODEL	облачный TTS	Piper через OpenAI-совместимый сервер

Без LLM_BASE_URL приложение работает в режиме заглушки: заявитель отвечает по фактам сценария правилами, проверки, которым нужна модель, помечаются «не применимо». Текущие адреса видны администратору на странице «Состояние».

Резервные копии и восстановление

См. admin.md.

Безопасность и персональные данные

Тренажёр работает в изолированном контуре учебного центра. Он обрабатывает ФИО обучающихся и преподавателей, результаты обучения и — при голосовых звонках — записи голоса. Это персональные данные по 152-ФЗ, поэтому защита заложена в устройство системы, а не добавлена сверху.

Доступ

Мера	Как сделано
Вход по логину и паролю при каждом входе	Сессия хранится в базе, в браузере — только случайный токен в cookie <code>HttpOnly, SameSite=Lax, Secure</code> при HTTPS. В базе лежит не сам токен, а его SHA-256
Парольная политика	Не короче 8 символов, буквы и цифры; пароли хранятся как <code>bcrypt</code> -хеш
Защита от подбора	После 5 неудачных попыток учётная запись закрывается на 15 минут, каждая попытка пишется в журнал
Истечение сессии	Через <code>SESSION_HOURS</code> часов; при блокировке пользователя или смене пароля все его сессии удаляются сразу
Роли	Администратор, преподаватель, обучающийся. Каждая страница и каждый API-запрос проверяет роль на сервере; обучающийся видит только свои результаты, преподаватель — свои занятия
Публичные адреса	Без входа доступны только страница входа и <code>GET /api/health</code> , который не отдаёт никаких данных

Журнал аудита и сбоев

Входы и неудачные попытки, блокировки, изменения пользователей, настроек и сценариев, подтверждение и правка оценок преподавателем (кто, когда, было → стало), резервные копии, ошибки сервера. Журнал хранится не меньше 6 месяцев, выгружается в CSV. Подробно — `admin.md`.

Шифрование

Внутри класса трафик идёт по HTTPS через Caddy с собственным центром сертификации (`tls internal`), на публичном стенде — с сертификатом Let's Encrypt. Без HTTPS браузер не даёт микрофон, поэтому голосовые звонки работают только в защищённом канале. База данных не публикуется наружу контейнерной сети.

Модели ИИ и данные

- В контуре все модели (языковая, распознавание и синтез речи) ставятся локально, данные не покидают сервер. Адреса моделей — настройка, а не код.
- На демо-стенде используется облачный API. Туда уходят только учебные реплики вымышленных заявителей и тексты карточек из билетов. Реальных вызовов 112 в системе нет.
- ИИ не выставляет итоговую оценку: он готовит черновик, оценка идёт в зачёт только после подтверждения преподавателем.

Резервные копии и восстановление

Ежедневная копия базы по расписанию и ручной запуск; хранение N дней; восстановление из консоли одной командой. Подробно – admin.md.

Соответствие 152-ФЗ и 149-ФЗ

- **Цель обработки** – обучение и аттестация персонала. Состав данных минимальный: ФИО, логин, роль, результаты попыток. Записи голоса хранятся только для разбора занятия.
- **Место обработки** – сервер учебного центра в РФ. Внешних передач нет, кроме облачной модели на демо-стенде.
- **Доступ** ограничен ролями, все действия журналируются.
- **Удаление** – администратор блокирует учётную запись. Записи журнала аудита хранятся установленный срок, затем удаляются автоматически.
- Оператор данных – учебный центр, он оформляет согласия слушателей по своему локальному акту. Система даёт для этого технические меры: разграничение доступа, журнал, шифрование, резервное копирование.

Производительность и нагрузка

Замеры сделаны на production-сборке в Docker (тот же образ, что на стенде) на ноутбуке с Intel Core i5-12450H (12 потоков) и 16 ГБ ОЗУ. База PostgreSQL 16 в соседнем контейнере. Скрипт — `scripts/loadtest.ts`, его можно повторить на любом сервере.

Требования ТЗ и результат

Требование	Как мерили	Результат
100 пользователей, отклик ≤ 2 с	100 виртуальных учеников: у каждого своё место ДДС с потоком карточек, опрос состояния места и ленты раз в 1,5 с плюс страница кабинета, 45 с	p95 0,69 с , p99 0,94 с, ошибок 0, 132 запроса/с
≥ 20 одновременных сессий	30 рабочих мест ДДС, у каждого свой поток карточек, опрос раз в секунду, 60 с	p95 0,23 с , ошибок 0
≥ 100 записей/с в БД	5000 вставок в журнал пачками по 50	≈ 5300 записей/с
Отчёт ≤ 30 с	отчёт занятия, выгрузка CSV, сводные отчёты	0,03–0,07 с

Команды:

```
pnpm exec tsx scripts/loadtest.ts --base http://<сервер>:3000 --users 30 --setup dds
--think 1000 --duration 60 --paths /api/dds/state,/api/dds/feed
pnpm exec tsx scripts/loadtest.ts --base http://<сервер>:3000 --users 100 --setup
dds --think 1500 --duration 45 --paths /api/dds/state,/api/dds/feed,/student
pnpm exec tsx scripts/loadtest.ts --db-writes 5000
```

Что для этого сделано

- **Несколько процессов приложения на одном порту** (`cluster.cjs`, переменная `APP_WORKERS`, по умолчанию по числу ядер, но не больше 4). Один процесс Node обрабатывает около 60 запросов опроса в секунду. С четырьмя процессами пропускная способность выросла в 2,4 раза: p95 при 100 рабочих местах упал с 2,4 с до 0,69 с. Фоновые задачи (резервные копии, чистка журнала, сброс демо) выполняет только один процесс.
- **Поток карточек места ДДС двигается не чаще раза в 2 секунды на место**, а счётчики и телефон читаются на каждом опросе. Для таймеров 30 секунд и 3 минуты такой точности хватает с запасом.
- **Пул соединений с базой** — 15 на процесс, всего 60 при лимите PostgreSQL в 100.
- Приложение не хранит состояние в памяти между запросами: сессии, очередь карточек и таймеры лежат в базе. Поэтому процессы и серверы можно добавлять без переделок.

Локальные модели

Замер языковой, распознающей и синтезирующей модели на процессоре и на видеокарте будет добавлен после установки локальных моделей. Сейчас демо-стенд использует облачный API, а без модели тренажёр работает на правилах: см. [methods.md](#).

Демо-стенд

Стенд должен проработать всю экспертизу без вмешательства. После срока сдачи его нельзя обновлять, поэтому всё, что может сломаться за две недели, закрыто заранее.

Что нужно

- VPS в РФ: 4 vCPU, 8 ГБ ОЗУ, 40 ГБ диска, Ubuntu 24.04.
- Домен или поддомен с А-записью на IP сервера.
- Ключ облачного ИИ-сервиса с OpenAI-совместимым API. В кабинете провайдера — лимит трат и бюджет с запасом на весь срок экспертизы, ключ без срока истечения.

Установка

```
## на сервере
curl -fsSL https://get.docker.com | sh
git clone <адрес репозитория> trainer && cd trainer
cp .env.example .env
## в .env: DEMO_MODE=true, LLM_* / STT_* / TTS_* — адреса и ключи облачного сервиса
SITE_ADDRESS=trainer.example.ru docker compose --profile app --profile https up -d -
-build
```

Проверка: `https://trainer.example.ru` открывает экран входа, `https://trainer.example.ru/api/health` отвечает `{"ok":true}`.

Почему стенд проживёт без присмотра

Риск	Что сделано
Упал процесс или перезагрузился сервер	<code>restart: always</code> у всех сервисов, проверка здоровья приложения и базы
Логи забили диск	ограничение логов контейнеров: 5 файлов по 10 МБ
Резервные копии забили диск	копии хранятся заданное число дней, старые удаляются планировщиком
Журнал аудита разросся	хранится 190 дней, дальше чистится планировщиком
Проверяющие заблокировали демо-учётки или сменили им пароль	при <code>DEMO_MODE=true</code> демо-учётки не блокируются, не закрываются после неверных паролей и не меняются
Проверяющие видят мусор друг друга	тренировки без занятия у каждого личные; каждую ночь в 04:30 МСК стенд возвращается к исходному виду — занятия, учётки и правки проверяющих удаляются, демо-занятия с попытками строятся заново
Кончились деньги на ключе ИИ	бюджет с запасом и лимит трат у провайдера; при ошибке модели заявитель продолжает отвечать по фактам сценария

Риск	Что сделано
Истёк сертификат	Caddy продлевает сертификат Let's Encrypt сам

Мониторинг

Внешняя проверка доступности (любой бесплатный сервис мониторинга) раз в 5 минут опрашивает `/api/health` и присылает уведомление владельцу. Уведомление — повод проверить сервер, но не обновлять стенд после срока сдачи.

Библиотеки и модели

Список собран автоматически командой `node scripts/licenses.mjs` из зависимостей production-сборки.

Модели и сервисы

Компонент	Назначение	Лицензия
Qwen2.5 7B / 3B Instruct	языковая модель в контуре (Ollama, llama.cpp)	Apache 2.0 (7B), Qwen Research (3B)
faster-whisper + Whisper large-v3-turbo / small	распознавание речи в контуре	MIT
Piper TTS + русские голоса	синтез речи в контуре	MIT (движок); лицензия голоса указана в карточке голоса
Caddy	HTTPS и собственный центр сертификации	Apache 2.0
PostgreSQL 16	база данных	PostgreSQL License

Библиотеки приложения (102)

Пакет	Версия	Лицензия
@babel/code-frame	7.29.7	MIT
@babel/compat-data	7.29.7	MIT
@babel/core	7.29.7	MIT
@babel/generator	7.29.8	MIT
@babel/helper-compilation-targets	7.29.7	MIT
@babel/helper-globals	7.29.7	MIT
@babel/helper-module-imports	7.29.7	MIT
@babel/helper-module-transforms	7.29.7	MIT
@babel/helper-string-parser	7.29.7	MIT
@babel/helper-validator-identifier	7.29.7	MIT
@babel/helper-validator-option	7.29.7	MIT
@babel/helpers	7.29.7	MIT
@babel/parser	7.29.9	MIT
@babel/template	7.29.7	MIT
@babel/traverse	7.29.8	MIT
@babel/types	7.29.8	MIT
@img/colour	1.1.0	MIT

Пакет	Версия	Лицензия
@img/sharp-libvips-linux-x64	1.3.3	LGPL-3.0-or-later
@img/sharp-libvips-linuxmusl-x64	1.3.3	LGPL-3.0-or-later
@img/sharp-linux-x64	0.35.4	Apache-2.0
@img/sharp-linuxmusl-x64	0.35.4	Apache-2.0
@jridgewell/gen-mapping	0.3.13	MIT
@jridgewell/remapping	2.3.5	MIT
@jridgewell/resolve-uri	3.1.2	MIT
@jridgewell/sourcemap-codec	1.6.0	MIT
@jridgewell/trace-mapping	0.3.31	MIT
@next/env	16.3.6	MIT
@next/swc-linux-x64-gnu	16.3.6	MIT
@next/swc-linux-x64-musl	16.3.6	MIT
@prisma/client	6.19.3	Apache-2.0
@prisma/config	6.19.3	Apache-2.0
@prisma/debug	6.19.3	Apache-2.0
@prisma/engines	6.19.3	Apache-2.0
@prisma/engines-version	7.1.1- 3.c2990dca591cba766e3b7ef5d9e8a84796e47ab7	Apache-2.0
@prisma/fetch-engine	6.19.3	Apache-2.0
@prisma/get-platform	6.19.3	Apache-2.0
@standard-schema/spec	1.1.0	MIT
@swc/helpers	0.5.23	Apache-2.0
@types/node	22.20.4	MIT
baseline-browser-mapping	2.11.26	Apache-2.0
bcryptjs	3.0.3	BSD-3-Clause
browserslist	4.29.1	MIT
c12	3.1.0	MIT
caniuse-lite	1.0.30001812	CC-BY-4.0
chokidar	4.0.3	MIT
citty	0.1.6, 0.2.2	MIT
client-only	0.0.1	MIT
confbox	0.2.4, 0.3.1	MIT

Пакет	Версия	Лицензия
console	3.4.2	MIT
convert-source-map	2.0.0	MIT
debug	4.4.3	MIT
deepmerge-ts	7.1.5	BSD-3-Clause
defu	6.1.7	MIT
dequal	2.0.3	MIT
destr	2.0.5	MIT
detect-libc	2.1.2	Apache-2.0
dotenv	16.6.1	BSD-2-Clause
effect	3.21.0	MIT
electron-to-chromium	1.5.439	ISC
empathic	2.0.0	MIT
escalade	3.2.0	MIT
exsolve	1.1.1	MIT
fast-check	3.23.2	MIT
gensync	1.0.0-beta.2	MIT
giget	2.0.0	MIT
jiti	2.7.0	MIT
js-tokens	4.0.0	MIT
jsesc	3.1.0	MIT
json5	2.2.3	MIT
lru-cache	5.1.1	ISC
ms	2.1.3	MIT
nanoid	3.3.19	MIT
next	16.3.6	MIT
node-fetch-native	1.6.7	MIT
node-releases	2.0.57	MIT
nypm	0.6.10	MIT
ohash	2.0.12	MIT
pathe	2.0.3	MIT
perfect-debounce	1.0.0	MIT
picocolors	1.1.1	ISC

Пакет	Версия	Лицензия
pkg-types	2.3.3	MIT
postcss	8.5.23	MIT
prisma	6.19.3	Apache-2.0
pure-rand	6.1.0	MIT
rc9	2.1.2	MIT
react	19.2.8	MIT
react-dom	19.2.8	MIT
readdirp	4.1.2	MIT
scheduler	0.27.0	MIT
semver	6.3.1, 7.8.5	ISC
sharp	0.35.4	Apache-2.0
source-map-js	1.2.1	BSD-3-Clause
styled-jsx	5.1.6	MIT
swr	2.5.1	MIT
tinyexec	1.3.1	MIT
tslib	2.8.1	0BSD
typescript	5.9.3	Apache-2.0
undici-types	6.21.0	MIT
update-browserslist-db	1.3.3	MIT
use-sync-external-store	1.7.0	MIT
yallist	3.1.1	ISC
zod	4.6.5	MIT